
OBJECT ORIENTED PROGRAMMING (CSSE7023)

ASSIGNMENT 2 — SEMESTER 1, 2026

SCHOOL OF ELECTRICAL ENGINEERING AND COMPUTER SCIENCE

THE UNIVERSITY OF QUEENSLAND

Due May 22nd 17:00 AEST

I hear and I forget, I see and I remember, I do and I understand.
— Confucian Aphorism

Revision 1.0.0

Overview This assignment delivers practical experience in maintaining and re-working parts of a pre-existing Java project as part of a larger development team, working on re-factoring and enhancing different aspects of the code. You will be assessed on your ability to:

- develop a program that complies with the specification and conforms to the style conventions of the course,
- implement Java file I/O and file management,
- develop a graphical user interface using the Model-View-Controller (MVC) paradigm,
- develop JUnit tests that can detect faults in class implementations, and
- use and critically assess the value of AI tools in your software development role.

Background In Queensland, the assessment of secondary school “senior” (Years 11 & 12) students is controlled by the Queensland Curriculum and Assessment Authority (QCAA). Senior school students are required to take up to six different subjects for their final two years of schooling. That is, senior school students take six subjects over four semesters. For each subject, each semester is called a “unit” of that subject – thus, senior school students take six “units” of subjects every semester.

Assessments can be internal assessments (marked by the school staff) or external examinations (marked by the QCAA). External exams are conducted in such a manner that all students in Queensland sit the same exam at the same time on the same day, this requires scheduling across the State by the QCAA, and all schools must follow the QCAA schedule for examinations. Because of this, schools have no control over when an exam is to occur and must allocate all students undertaking the relevant subject to an appropriate exam venue within their school, at the same time as every other student in every other school in the State.

A Java program has been developed for a school to use, to schedule students to assigned exam sessions, using a time schedule dictated by the QCAA; and arranging those students within the school’s available exam venues, providing the seating plans within those venues, in accordance with the school’s requirements.



Figure 1: An exam venue.

Students are seated in alphabetical order by surname starting from the desk in the front left corner of the room, as seen when facing towards the front of the room, filling the desks in alphabetical order from front desk to the rear desk in each “column” of desks, before moving to the next column of desks and again filling from front to rear in each column, from left to right across the room.

For example, a venue with 5 rows of 6 desks (6 columns), could be represented like this:

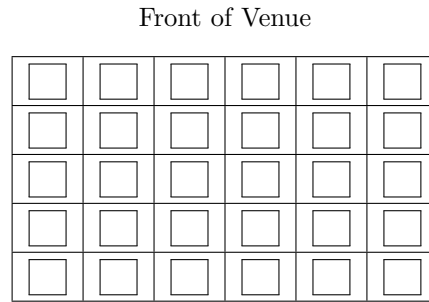


Figure 2: A representation of the desk layout in an exam venue.

The students would fill the venue in the following order, alphabetical by surname:

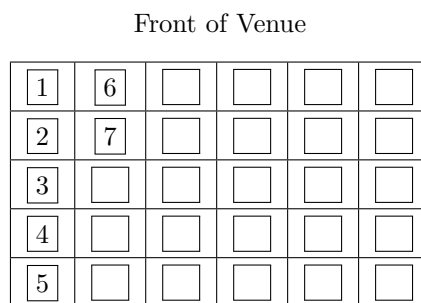


Figure 3: The order in which students are seated in an exam venue.

An exam venue can consist of one more (joined) rooms. A room can only be used for one exam venue at any one time. For example, say three interconnecting rooms (separated by collapsable walls) are “R1”, “R2”, and “R3”. Any of these can be an exam venue by itself, say “V1” can hold 25 desks in a 5x5 configuration, “V2” can hold 25 desks in a 5x5 configuration, and “V3” can hold 25 desks in a 5x5 configuration. However, these rooms can also be joined. Say the venue “V1+V2” is made by joining rooms R1 and R2 together and V1+V2 can hold 55 desks in a 5x11 configuration (slightly more space than two separate rooms because the wall has been removed). However, this now means that the venues V1 and V2 are no longer available because rooms R1 and R2 have been used to make the venue called “V1+V2”.

Exam venues can come in any configuration and are simply specified by the number of rows (front to rear) and the number of columns (left to right) of desks in the venue. Note that rooms do not have desk configurations recorded, only exam venues. (Rooms can normally hold many more desks when they are not set up as exam venues, but our model does not need that information – only the desk configuration of exam venues.)

For example, a venue with 5 rows x 9 columns of desks:

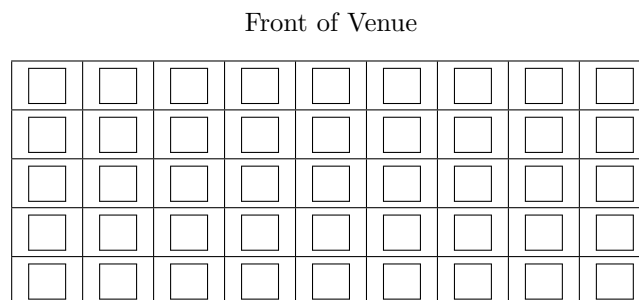


Figure 4: A larger exam venue (two rooms joined side-by-side).

Or maybe, 15 rows x 4 columns:

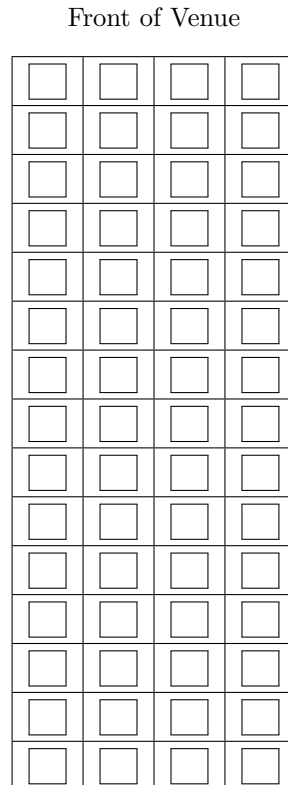


Figure 5: A larger exam venue (two rooms joined front-to-back).

Given the venues set up in this manner, your system must then determine for each scheduled exam in the QCAA schedule: which students undertake that subject, and therefore that exam, and then allocate students in alphabetical order by surname to the available desks starting on the left hand side of the room from the front left desk to the rear left desk, and then in successive columns, only using available desks in the exam room.

This process is further complicated by the fact that the QCAA will schedule multiple smaller subjects to have exams at the same time. You must allocate the students undertaking all such subjects into the available desks in the exam venue at the same time.

There are a number of conditions In varying order of importance that must be satisfied when doing this.

- Firstly, when more than one exam is occurring at the same time, if a student his undertaking more than one of the subjects involved in those exams this must be flagged as a clash, as an individual student cannot undertake multiple exams at the same time.
- Secondly, where possible, students sitting multiple subjects in the same venue should be separated by as many empty desks as possible.
- Next, in any exam, students should be separated as much as possible within the exam venue. Separating columns of students is more important than separating roles of rows of students, as it is easier to supervise wandering eyes when students are laterally further apart; distances front to rear are not as important as side to side.

The final step after allocating the students to desks, is to display the desk assignment, in tabular form, exactly as you would view the venue, like this:

Venue V1					
Desk 1: Surname1 Given1 I.	Desk 6: Surname6 Given6 I.	Desk 11: Surname11 Given11 I.	Desk 16: Surname16 Given16 I.	Desk 21: Surname21 Given21 I.	Desk 26: Surname26 Given26 I.
Desk 2: Surname2 Given2 I.	Desk 7: Surname7 Given7 I.	Desk 12: Surname12 Given12 I.	Desk 17: Surname17 Given17 I.	Desk 22: Surname22 Given22 I.	Desk 27: Surname27 Given27 I.
Desk 3: Surname3 Given3 I.	Desk 8: Surname8 Given8 I.	Desk 13: Surname13 Given13 I.	Desk 18: Surname18 Given18 I.	Desk 23: Surname23 Given23 I.	Desk 28: Surname28 Given28 I.
Desk 4: Surname4 Given4 I.	Desk 9: Surname9 Given9 I.	Desk 14: Surname14 Given14 I.	Desk 19: Surname19 Given19 I.	Desk 24: Surname24 Given24 I.	Desk 29: Surname29 Given29 I.
Desk 5: Surname5 Given5 I.	Desk 10: Surname10 Given10 I.	Desk 15: Surname15 Given15 I.	Desk 20: Surname20 Given20 I.	Desk 25: Surname25 Given25 I.	Desk 30: Surname30 Given30 I.

Figure 6: Sample seating allocation (where I. represents the initial of the student's middle name).

Existing CLI program The existing program is **NOT** an example of good OO programming — or even good programming in general — having a massive “Catalogue” class containing all the school’s students, all the subjects, subject units, exams, exam venues and room data in a single hard-coded class of data, along with poor use of objects, a hard-to-use command-line interface (CLI), hard-to-read text-only output, a number of minor coding errors and a few *significant design faults*, that actually make the original coding even more discordant.

You are part of a team performing maintenance and upgrades on this software project. The client now wants a Graphical User Interface (GUI) application to interact with, when allocating exams to venues, and they want the app to check, and *where possible fix*, the various inputs to meet the specifications. They also want to be able to load all of the data in from a file (and save it too) and not have the data all hardcoded into the program.

Task - Refactoring the Exam Block project In this assignment, you will update, modify and extend code written by others. You will modify and extend a version of the Exam Block packages, written by other programmers who have already refactored some parts of the original command-line program. You will develop new versions of the remaining missing Exam Block classes, in order to extend the functionality, to read and interpret (parse) input data into a new registry object (replacing the cumbersome hard-coded Catalogue from the command-line version) and to provide a GUI for an interactive user to update the exam allocations. This will not only involve developing the new classes, but will also require you to integrate with the new design that has been developed and the provided code written by others.

Your submitted implementation will read and interpret initial input data in a supplied input text file, correcting any incorrect data formats, as required. Your submission will be tested against a number of input text files, not only the one included in the provided code.

Your implementation will also write output data files in the correct format to be used again later as input for the system to continue or modify the allocations.

Your new code will be tested against this assignment specification document and the supplied Javadoc. **You are expected to test your code yourself.** *Any provided Gradescope tests are only to confirm that your submission compiles.* The assessment tests will be conducted after all assignment submissions are complete (including all late submissions).

You will also write your own (assessed) JUnit tests for some of the classes in the implementation and include these in your submission. You are encouraged to write tests for *all* your classes and for the GUI part of your implementation as well, but you do not have to submit these. Only the tests for specific classes are to

be submitted. Any test you write for other aspects of the project are not to be included in your submission. The required tests that you have designed and submitted will be assessed for their effectiveness against correct implementations of the code, as well as a number of incorrect implementations specifically written by the staff to test your tests.

The required tests are for the **Subject**, **Unit**, **Venue**, **Session**, and **Student** classes. Your submitted test classes must be named **SubjectTest**, **UnitTest**, **VenueTest**, **SessionTest**, and **StudentTest**. Each class may contain whatever tests you deem appropriate, but all the tests for each class must be in a single Java class file.

The Javadoc describes the classes that your assignment must implement. A number of classes have not been included in the provided code. You must develop and submit new classes for these. Don't forget to include complete Javadoc within your new classes to match and enhance the provided Individual Assignment 2 Javadoc.

Download the **bundle** scripts to submit your complete assignment including your five test classes and your AI Usage Log and **refs.md**. **Note that the “provided.zip” includes the bundle scripts and the Javadoc.**

You will be assessed on your ability to

- conform with the required files and public class signatures,
- develop a working program that complies with the new specification,
- develop code that conforms to the style conventions of this course,
- develop a working GUI using the Model-View-Controller (MVC) paradigm,
- correctly implement Java file I/O and file management,
- develop JUnit tests that can detect faults in class implementations, and
- critically assess the use of AI tools in your software development role.

Generative Artificial Intelligence (AI/GenAI) Being familiar with AI coding tools is now expected in industry because such tools significantly increase developer productivity, improve code quality, and accelerate problem-solving, making them a standard part of modern professional software development.

AI (e.g. Claude, ChatGPT, Copilot, etc.) and Machine Translation (MT) are emerging tools that may support students in completing this assessment task. Students must use AI tools, at least in some way, and record and submit the required **AI Usage and Decision Log** as part of this assignment.

If you do not already have your own preferred AI tools, the University of Queensland prefers Microsoft Copilot for reasons of data security within the University of Queensland's separate Microsoft 365 tenant.

Please note that submitting an assignment that was *entirely* written by AI, with no student content or value added, correctly referenced throughout, clearly identifying that all code was written by AI, is not plagiarism, but since there is no student content or value, it **has no academic merit** and so would receive zero marks.

To pass this assessment, students may be required to demonstrate detailed comprehension of their submission independent of any AI and MT tools, as part of the assignment marking process.

All submitted code will be subject to electronic plagiarism and collusion detection. The assignments are to be worked on individually and must be your own work except where the use of code written or provided by other entities is explicitly permitted by the assignment specification, and any such code is referenced in the manner required in the assignment specification.

This is a learning exercise and you will harm your learning if you use AI tools inappropriately. Remember, you will be required to write code, by hand, in the final exam.

Plagiarism All work on this assignment is to be your own individual work (including your own use of AI tools). By submitting the assignment you are claiming it is your own work. You *may* discuss the overall general design of the application with other students. Describing details of how you implement your design with another student is considered to be **collusion** and will be counted as plagiarism.

Do **not** show other students your code or your *AI Usage and Decision Log*. Do **not** look at other students' code or other students' *AI Usage and Decision Logs*. Do **not** copy fragments of code from other students. Code supplied by course staff (from *this* semester) is acceptable, but must be clearly acknowledged as described in the next paragraph. You do not need to acknowledge the provided code that was supplied with this specification – it is part of the specification and is not included in the academic integrity checks. Code generated by third-party tools is also acceptable, but **must** also be clearly acknowledged, see *Generative Artificial Intelligence* above.

You may find ideas of how to solve problems in the assignment through external resources (e.g. StackOverflow, textbooks, ...). If you use these ideas in designing your solution you **must** cite them. To cite a resource, provide the full bibliographic reference for the resource in file called `refs.md`.

The `refs.md` file **must** be in the root folder of your project.
For example:

```
1 > cat refs.md
2 [1] E. W. Dijkstra, "Go To Statement Considered Harmful," _Communications of the ACM_,
3     vol 11 no. 3, pp 147-148, Mar. 1968. Accessed: Mar. 22, 2026. [Online]. Available:
4     https://www.cs.utexas.edu/users/EWD/transcriptions/EWD02xx/EWD215.html
5 [2] B. Liskov and J. V. Guttag, _Program development in Java: abstraction,
6     specification, and object-oriented design_. Boston: Addison-Wesley, 2001.
7 [3] T. Hawtin, "String concatenation: concat() vs '+' operator," stackoverflow.com,
8     Sep. 6, 2008. Accessed: Mar. 23, 2026. Available:
9     https://stackoverflow.com/questions/47605/string-concatenation-concat-vs-operator
10 >
```

In the code where you use the idea, cite the reference in a comment. For example:

```
1 /**
2  * What method1 does.
3  * [1] Used a method to avoid gotos in my logic.
4  * [2] Algorithm based on section 6.4.
5  */
6 public void method1() ...
7
8 /**
9  * What method2 does.
10 */
11 public void method2() {
12     System.out.println("Some " + "content") // [3] String concatenation using + operator
13 }
```

You must be familiar with the university's policy on plagiarism.

<https://uq.mu/r1553>

If you have questions about what is acceptable, please ask course staff.

SPECIFICATION

This document specifies the assignment conditions and assessment. Read this assignment specification *carefully*. The software specification is provided in the form of Javadocs.

- Implement the classes **exactly** as described in the Javadocs.
- Read the Javadocs carefully and understand the program specification **before** programming.
- Do not change the public specification in **any** way, **including** changing the names of, or adding additional, public classes, interfaces, methods, or fields.
- You are encouraged to add additional **private** members, classes, or interfaces as you see fit.
- JUnit 4 test cases that you submit must only test the public and protected methods as specified in the Javadocs. They must not rely on any other members, classes, or interfaces you may have added to the classes being tested. You may create private members and other classes in your test code.

GETTING STARTED

To get started, read this document right to the end – do not jump to the Javadoc or start coding before you read this assignment specification – there are important assessment details following.

After you have finished this document, you can download the Javadoc specification from **BlackBoard** (**Assessment** → **Individual Assignment 2**), unpack it and open the `index.html` in your favourite browser, or access it ready-to-view at the link below.

<https://csse7023.uqcloud.net/assessment/assign2/docs/>

Next, download the provided code from **BlackBoard**. Extract the archive into the directory you want to work from and open it with *IntelliJ*.

GRADING

Seven aspects of your solution will be considered in grading your submission. These are *conformance*, an *automated functionality test*, an *automated style check*, the assessment of your *JUnit tests*, a manual *human readable style* assessment, the manual *GUI functionality* assessment, and your *AI Usage and Decisions Log* assessment:

1. *Conformance*: You are only coding **part** of this program. Other parts are created by others. Your classes must integrate exactly into the overall program. You must implement the specified classes, and only the specified classes as public classes, exactly as described in the Javadocs. Your public methods must have the exact signature specified. (You may add as many private “helper” methods as you see fit, but these must be private and not visible outside their class.) The submission platform will check that your submission matches the expected signatures.
2. *Automated functionality test*: the classes that you must implement will have a number of JUnit unit tests associated with them that **we** will use to test your implementation. The percentage of test cases that pass will be used as part of your grade calculation. Classes may be weighted differently depending on their complexity.
3. *Automated style check*: Your grade for automated style checking is based on the number of style violations identified by the *Checkstyle* tool¹. It will be run in the same environment as the JUnit automated functionality tests. Multiple style violations of the same type will **each** count as **additional** violations.

Note: There is a plug-in available for *IntelliJ* that will highlight style violations in your code. Instructions for installing this plug-in are available in the Java Programming Style Guide on BlackBoard (Learning Resources → Guides). If you correctly use the plug-in and follow the style requirements, it should be relatively straightforward to get high grades for this section.

4. *JUnit test cases*: **your** JUnit test cases will be assessed by testing both correct *and* faulty implementations. The percentage of implementations that are appropriately identified as correct or faulty will be used as part of your grade calculation.
5. *Manual style check*: as for Assignment 1, the style and structure of your code will also be assessed by course staff. Your performance on these criteria will also be used as part of your grade calculation. It is therefore critically important that you read and understand the feedback for this part on Assignment 1, as soon as it is made available, so that you can address any issues in this assignment. See Appendix B for criteria that will be used to assess the readability of your code.
6. *Manual GUI functionality test*: to ensure that the look and feel of your GUI implementation is the same as the implementation demonstrated in lectures, and to ensure that the **Open/Load** and **Save** menu options are properly invoked by the GUI, a small scenario with a number of steps in it will be manually executed by course staff. The faults identified during these steps will be used as part of your grade calculation.
7. *AI Usage and Decision Log*: Course staff will grade the required **AI Usage and Decision Log** as part of this assignment. This will assess both the Transparency and Professional development sections of your log.
 1. Transparency – You are required to accurately record AI use and non-use in line with university expectations.
 2. Professional development – The log helps you develop responsible, intentional judgement about when and how AI tools are used.

NOTE that you **must use the Individual Assignment 2 version**,

`CSSE7023_UsageandDecisionLog_Assignment2_v1.0`, **not** the Assignment 1 version of the log for this assignment! See the first page of the log form for the requirements of your log.

¹The latest version of the course *Checkstyle* configuration can be found at <http://csse7023.uqcloud.net/checkstyle.xml>. See the *Style Guide* on BlackBoard for instructions on how to use it in *IntelliJ*.

Four aspects of assessment will be performed automatically in a Linux environment: conformance checks, execution of (*our*) JUnit functionality test cases, running *your* JUnit test cases on correct and faulty implementations, and automated style check using *Checkstyle*. The environment will not be running Windows, and neither *IntelliJ* nor *Eclipse* (or any other IDE) will be involved. OpenJDK 25 with the JUnit 4 library will be used to compile and execute your code and tests. To prevent infinite loops, or malicious code, from slowing down Gradescope, any test that takes longer than 10 seconds to execute will be killed and identified as failing. All tests should execute in a small fraction of a second. Any test taking longer than a second to execute indicates faulty logic or malicious code. Similarly, any of your JUnit test cases that take longer than 20 seconds to execute on one of the correct/faulty implementations, or that consume more memory than is reasonable, will be stopped.

IDEs like *IntelliJ* provide code completion hints. When importing Java libraries they may suggest libraries that are not part of the standard library. These will **not** be available in the test environment and your code will **not** compile. When uploading your assignment to Gradescope, **ensure** that Gradescope says that your submission was compiled successfully.

Your code must compile.
If your submission does not compile, you will receive no marks.

SUBMISSION

Submission is via Gradescope. Submit your code to Gradescope *early and often*.

Unlike Individual Assignment 1, for Individual Assignment 2, Gradescope will only give you a *very small* amount feedback on your code every time you submit, but it does include all the conformance and automated style tests. Most importantly, it confirms that your code compiles and runs.

It will **not** do all of the automatic functionality testing before assessment like Individual Assignment 1 did. **You must test your code yourself!** Gradescope will not execute the full functionality assessment before the assignment closes. You are expected to test that your code correctly follows the specification yourself. After the assignment closes (including any extensions and any late submissions), Gradescope will then execute the full functionality assessment and then all results (automated and manual grading marks) will be made available in Gradescope and the raw assignment marks, “*Individual Assignment 2 Submission*” (before any late penalties), will be available in Blackboard. Any late penalties will be applied before final assignment marks, “*Individual Assignment 2 Total*”, are published in Blackboard.

What to Submit Your submission **must** have the following internal structure:

- `src/` Folders (packages) and `.java` files for classes that you modified or created for this assignment.
- `test/` Folders (packages) and `.java` files for the JUnit tests that are required for this assignment.
- `refs.md` file containing the references for any citations in your code.
- `CSSE7023_UsageandDecisionLog_Assignment2_v1.0.docx`

Included in the root directory of the provided code are the files `bundle.sh` and `bundle.cmd`. For MacOS and Unix users, run `$bash bundle.sh` (or just `./bundle.sh`) to execute it. For Windows users, double-click or run the `.\bundle.cmd` file to execute it. This will create a `submission.zip` file for you to upload to Gradescope.

You can create the submission zip file yourself using a zip utility. If you do this, **ensure** that you do not **miss** any files or directories. Also **ensure** that you do not **add** any extra files. We recommend using the provided `bundle` scripts.

Ensure that your classes and interfaces **correctly** declare the package they are within. For example, `Student.java` should declare `package examblock.model;`

Only submit the `src` and `test` folders, `refs.md` and your AI log file in the root directory of your project. **Do not** submit **any** other files (e.g. no `.class` files or IDE files).

ASSESSMENT POLICY

Late Submission You must submit your code *before* the deadline. Code that is submitted after the deadline will receive a late penalty as described in the Course Profile. The submission time is determined by the time recorded on the Gradescope server. A submission is not recorded as being received until uploading your files completes. Attempting to submit at the last minute may result in a late submission.

You may submit your assignment to Gradescope as many times as you wish before the due date. If a misconduct case is raised about your submission, a history of regular submissions to Gradescope, which demonstrate progress on your solution, could support your argument that the work was your own. However, that history may also show when a submission is plagiarised and subsequently changed to attempt to hide the plagiarism.

You are *strongly* encouraged to submit your assignment on time, or by your assigned revised deadline if the School of Electrical Engineering and Computer Science has granted you an extension.

Experience has demonstrated that most students who submit their assignments late lose more grades due to the late penalties than they gain by making improvements to their work.

Extensions If an unavoidable disruption occurs (e.g. illness, family crisis, etc.) you should consider applying for an extension. Please refer to the following page for further information.

<https://uq.mu/r1551>

All requests for extensions must be made via my.UQ, *before* the submission deadline. Do not email the course coordinator or other course staff to request an extension.

Re-Grading If an *administrative error* has been made in the grading of your assignment, please contact the course coordinator (csse7023@uq.edu.au) to request this be fixed. For all other cases, please refer to the following page for further information.

<https://uq.mu/r1552>

CHANGE LOG

REVISION: 1.0.0

If it becomes necessary to correct or clarify the task sheet or Javadoc, a new version will be issued and an announcement will be made on the course Blackboard site. All changes will be listed in this section of the task sheet.

A CRITICAL MISTAKES

THINGS YOU MUST AVOID

This is being heavily emphasised here because these are critical mistakes which **must** be avoided.

Code may run fine locally on your own computer in *IntelliJ*, but it is **required** that it also builds and runs correctly when it is executed by the automated grading tool in Gradescope. Your solution needs to conform to the specification for this to occur.

- Files must be in the correct directories (**exactly**) as specified by the Javadoc. If files are in incorrect directories (*even **slightly** wrong*), you may lose grades for functionality in these files because the implementation does not conform to the specification.
- Files must have the correct package declaration at the top of every file. If files have incorrect package declarations (*even **slightly** wrong, such as incorrect capitalisation*), you may lose grades for functionality in these files because the implementation does not conform to the specification.
- You must implement the public and protected members **exactly** as described in the supplied documentation (**no** *extra public/protected members or classes*). Creating public or protected data members in a class when it is not specified will result in loss of grades, because the implementation does not conform to the specification.
 - You are *encouraged* to create private members as you see fit to implement the required functionality or improve the design of your solution.
- **Do not** change any of the provided code. Your submission must work with the provided code, as supplied. When you submit, any provided code is overwritten with a clean copy. If you accidentally change provided code in your working copy, you may find that your code only works on your machine and doesn't work when submitted.
- **Do not** import the `org.junit.jupiter.api` package. This is from JUnit 5 and may cause the JUnit tests to fail.
- Do not use **any** version of Java other than 25 when writing your solution. If you accidentally use Java features which are different in a version older than 25, then your submission may fail functionality tests. If you accidentally use Java features which are only present in a version newer than 25, then your submission may fail to compile.

B GRADING

AUTOMATED TESTING & CHECKING

Conformance, functionality, and style checking are assessed via an autograder, automated JUnit tests (*ours*), and the *Checkstyle* tool.

Your **conformance** marks will be 5% of the total marks, **LESS** the number of conformance violations identified. Five or more conformance violations will result in zero marks for this component.

Your **functionality** marks will be 15% of the total marks based on the percentage of unit tests you pass.

Your **automated Checkstyle** marks will be 10% of the total marks, **LESS** the number of style violations identified. Ten or more style violations will result in zero marks for this component.

Your **JUnit test** marking will be performed by running *your* JUnit test cases against correct and faulty implementations and count for 20%.

Manual **GUI functional** testing will be performed by course staff and count for 20%.

Manual **Code Style and Structure** testing will be performed by course staff, as described below, under **Code Style and Structure**, and count for 20%.

Finally, your **AI Log** will be assessed by course staff and count for 10%.

CODE STYLE AND STRUCTURE

The style and structure of your code will be assessed by course staff. Style and structure will be graded according to the provided rubric. The key consideration in grading your code style is whether the code is easy to understand. Code style will be assessed against the following criteria.

Readability

- Program Structure: Layout of code makes it easier to read and follow its logic. This includes using whitespace to highlight blocks of logic.
- Descriptive Identifier Names: Variable, constant, function, class and method names clearly describe what they represent in the program's logic. Do **not** use what is called the *Hungarian Notation* for identifiers. In short, this means do not include the identifier's type in its name (e.g. `item_list`), rather make the name meaningful. (e.g. Use `items`, where plural informs the reader it is a collection of items and it can easily be changed to be some other collection and not a list.)
- Named Constants: All non-trivial fixed values (literal constants) in the code are represented by descriptive named (symbolic) constants.

Documentation

- Comment Clarity: Comments provide meaningful descriptions of the code. They should not repeat what is already obvious by reading the code (e.g. `# Setting variable to 0.`). Comments should not be verbose or excessive, as this can make it difficult to follow the code.
- Informative Javadoc: Every class, method and member variable⁴ should have a Javadoc comment that explains its purpose. This includes describing parameters, return values, and potentially thrown exceptions so that others can understand how to use the method correctly.
- Description of Logic: All significant blocks of code should have a comment to explain how the logic works. For a small method, the logic should usually be clear from the code and Javadoc. For long or complex methods, each logical block should have an in-line comment describing its logic.

Design & Logic

- Single Instance of Logic: Blocks of code should not be duplicated in your program. Any code that needs to be used multiple times should be implemented as a method.
- Variable Scope: Variables should be declared locally in the method in which they are needed. Class variables are avoided, except where they simplify program logic.
- Control Structures: Logic is structured simply and clearly through good use of control structures (e.g. well-designed loops and conditional statements).
- Encapsulation: Classes are designed as self-contained entities with state and behaviour. Methods only directly access the state of the object on which they were invoked. Methods never update the state of another object.

RUBRIC

Criteria	Standard		
Readability	Advanced	Proficient	Developing
Program Structure	Whitespace & comments highlight all blocks of logic, making it easy to follow.	Whitespace & comments highlight some blocks of logic, decreasing readability at times.	Whitespace & comments are not used well, decreasing readability in several places.
Identifier Names	All identifier names are informative and well chosen, increasing readability of the code.	Most identifier names are informative, aiding code readability to some extent.	Several identifier names are not informative, detracting from code readability.
Symbolic Constants	All, non-trivial, constant values are informative and well named, symbolic constants.	Most, non-trivial, constant values are informative, symbolic constants.	Only some, non-trivial, constant values are informative, symbolic constants.
Documentation			
Comment Clarity	Almost all comments enhance the comprehensibility of the code. Comments never repeat information already apparent in the code, nor are they verbose.	A few comments are unnecessary to code comprehension. Or, a few comments are overly verbose, reducing the ease with which code can be understood.	Many comments are unnecessary to code comprehension. Or, some comments are overly verbose, reducing the ease with which code can be understood.
Informative Javadoc	All provided Javadoc is replicated or improved. Accurate & informative Javadoc is provided for all new methods & member variables.	All provided Javadoc is at least replicated. Accurate & informative Javadoc is provided for most new methods & member variables.	Some Javadoc is inaccurate, unclear or absent.
Description of Logic	All important or complex blocks of logic are clearly explained or summarised. No stating of the obvious.	Most important or complex blocks of logic are clearly explained or summarised. Almost no stating of the obvious.	Some blocks of logic are poorly explained or summarised. Or, some descriptions are verbose or confusing.
Design & Logic			
Single Instance of Logic	Almost no duplicate code. Additional well designed methods modularise your code.	Some code has been duplicated. You have added some methods to modularise your code.	Large amounts of code are duplicated.
Variable Scope	All member variables are necessary parts of the class' abstraction. None should have been local to methods. Class variables have been avoided or simplify logic.	A few member variables should be local variables. Or, there are a few unnecessary local variables in methods. Class variables have been avoided.	Some member variables are unnecessary or should be local variables. Class variables have been used like modular global variables.
Control Structures	Logic is simple and clear through good use of control structures.	A small number of control structures are unnecessarily complex.	Some poorly designed control structures (e.g. excessive nesting or branching, overly complex logic, multiple unnecessary exit points, ...).
Encapsulation	All classes are independent entities with private state and public behaviour. Methods only directly access their own object's state, never modifying another's.	Most classes are independent entities with private state and public behaviour. Methods rarely directly access or modify another object's state.	Some classes have non-private member variables. Some methods directly access or modify other objects' state.